

Code Review — Proiect Încapsulare

Feedback pe clasele Masina / MasinaEnc / Persoana / ContBancar

MyCodeSchool

Cum citești acest review

Proiectul e didactic, despre **încapsulare**: Masina (fără încapsulare, câmpuri public) în contrast cu MasinaEnc (cu private + getteri/setteri), plus Persoana și ContBancar. Feedback-ul e ordonat pe severitate: întâi bug-urile care fac codul să se comporte greșit, apoi problemele de încapsulare (chiar tema exercițiului), apoi stil.

Bug-uri reale — codul nu face ce pare

1. MasinaEnc.setAnFabricatie — folosește câmpul în loc de parametru

```
public void setAnFabricatie(int an){
    if(anFabricatie >= 1950 && anFabricatie <= 2025){    // ← verifică CÂMPUL, nu 'an'
        this.anFabricatie = anFabricatie;              // ← se atribuie pe el însuși
    }
```

Parametrul an nu e folosit niciodată. Câmpul pornește de la 0, deci 0 >= 1950 e mereu false -> metoda **nu setează niciodată nimic** și afișează mereu eroare.

Corect:

```
public void setAnFabricatie(int an){
    if(an >= 1950 && an <= 2025){
        this.anFabricatie = an;
    }
```

Este exact greșeala de „shadowing” / this — merită discutată explicit, e lecția de învățat aici.

2. MasinaService.aplicaReducere — împărțire de întregi

```
int pretRedus = (100 - procentReducere) / 100;    // pt 10% -> 90/100 = 0 (int!)
masini.get(i).pret *= pretRedus;                  // toate preturile devin 0
```

90 / 100 în int = 0. După reducere, **toate mașinile ajung la preț 0**. În plus, logica e greșită: înmulțește cu factorul în loc să scadă din preț.

Corect:

```
double factor = (100.0 - procentReducere) / 100.0;
masini.get(i).pret = (int)(masini.get(i).pret * factor);
```

3. ContBancar(String titular, double soldInitial) — nu setează titularul

```
public ContBancar(String titular, double soldInitial){
    if(titular != null){
        this.sold = soldInitial;    // ← lipsește this.titular = titular;
    }
}
```

Contul rămâne cu titular == null chiar dacă i-ai dat un nume valid. Apoi descriere() va zice „Titularul nu exista”.

4. Comparatie de String cu != / ==

ContBancar (setTitular, getSold):

```
if(titular != null && titular != "")    // ← != pe String compară referințe, nu conținut
```

Trebuie !titular.isEmpty() (sau !"".equals(titular)).

MasinaService (numarMasiniAutomate, numarMasiniManuale, afiseazaMasiniAutomate):

```
if(masini.get(i).modTransmisie == "Automata")    // ← == pe String
```

Aici „merge din întâmplare” pentru că sunt literali interniți în string pool — dar e exact tipul de bug care explodează mai târziu. Regula: **la String mereu .equals()**.

Probleme de încapsulare — chiar tema exercițiului

5. Constructorii din MasinaEnc ocolesc validarea și ignoră parametri

```
public MasinaEnc(String marca, String model, int kilometraj, int pret,
                 int anFabricatie, String modTransmisie){
    this.marca = marca;
    this.model = model;
    // kilometraj, pret, anFabricatie, modTransmisie ← primiti dar NICIODATA setati
}
```

În Main.ex05 se trimit -1 km, an 1853, „Hibrid” — toate invalide — dar nici nu sunt validate, nici măcar salvate. Ideea încapsulării e ca **constructorul să folosească setterele**:

```
public MasinaEnc(String marca, String model, int kilometraj, int pret,
                 int anFabricatie, String modTransmisie){
    this.marca = marca;
    this.model = model;
    setKilometraj(kilometraj);
    setPret(pret);
    setAnFabricatie(anFabricatie);
    setModTransmisie(modTransmisie);
}
```

6. Persoana.descriere() e private

Nu poate fi apelată din afară -> practic inutilă. Ar trebui public. Comparativ, în ContBancar e public — inconsistență.

7. Getterii amestecă responsabilități

MasinaEnc.getMarca() face și System.out.println, și return. Un getter ar trebui **doar** să întoarcă valoarea; afișarea e treaba apelantului. Persoana nu are deloc getteri.

8. getSold(String titular) din ContBancar — design confuz

Un „getter” care întoarce void, primește un parametru și nu compară cu titularul contului. Fie e getter real (double getSold()), fie o metodă afiseazaSold().

Minore / stil

- **ContBancar.setTitular** resetează sold = 0 la fiecare apel -> poate șterge banii dacă renumești titularul.
- **ContBancar.retrage**: suma < sold ar trebui suma <= sold (altfel nu poți retrage exact tot soldul).
- **MasinaEnc**: import java.sql.SQLOutput; — import nefolosit, șterge-l.
- **Persoana.setNume**: blacklist e case-sensitive („ana” da, „Ana” nu) — inconsistent cu setOras care face toLowerCase().
- **Main.ex05**: System.out.println(masinaEncapsed_1) fără toString() afișează app.MasinaEnc@1b6d3586. Ar merge un toString() care întoarce descriere().
- **Afișări marcă+model** (ex. afiseazaCeaMaiNouaMasina): concatenare fără spațiu -> BMWX3. Adaugă " ".
- **Masina cu câmpuri public**: intenționat, ca exemplu de „cum NU se face” în contrast cu MasinaEnc. E ok didactic.

Rezumat pentru Adrian

Cel mai important de corectat, în ordine:

1. **#1** setAnFabricatie — nu setează niciodată.
2. **#2** aplicaReducere — face toate prețurile 0.
3. **#3** ContBancar — titular rămâne null.
4. **#4** == / != la String.

Astea patru sunt bug-uri reale care fac codul să se comporte greșit. Restul (**#5–#8**) sunt fix miezul lecției de încapsulare: constructorii trebuie să treacă prin settere, iar getterii/setterii să aibă o singură responsabilitate.

Legătură cu materialul „Referință vs Instanță vs Instanțiere”: bug-ul #4 (== la String) și comportamentul `cont2 = cont1` din `Main.ex4` sunt exact conceptul de **referință vs conținut** — merită parcurse împreună.